

RAPPORT TECHNIQUE DE REPRISE DE PROJET

Je Demain (anciennement MakeGiver)

Plateforme communautaire de solutions adaptées aux personnes en situation de handicap

Technologie principale	Symfony 8.0 (PHP 8.4)
Base de données	MySQL 8.0
Frontend	Bootstrap 5.3, Twig, Stimulus, Turbo
Version du rapport	Mai 2025
Statut du projet	Pré-alpha, développement actif
Destination	Reprise par un groupe ou une tierce entité

Sommaire

1. Contexte et objectifs du projet
2. Architecture technique globale
3. Technologies utilisées et dépendances
4. Structure du projet et organisation des fichiers
5. Modèle de base de données
6. Fonctionnalités implémentées
7. Système d'authentification et rôles
8. Interface utilisateur et design
9. API externe et intégrations tierces
10. Points d'attention et travail restant
11. Instructions de mise en service

1. Contexte et objectifs du projet

Je Demain (dont le nom de code technique dans les fichiers est encore MakeGiver) est une plateforme web communautaire dont la vocation est de connecter trois types d'acteurs : les personnes en situation de handicap ou de dépendance (appelées ici Membres ou Patients), les Makers (bricoleurs, fabricants, makers au sens du mouvement maker), et les Soignants. L'idée centrale est simple : une personne ne trouvant pas d'aide adaptée à son quotidien peut exprimer un besoin technique, et un Maker peut proposer une solution concrète, souvent fabriquée via impression 3D ou prototypage.

Le projet est directement inspiré du mouvement open-source hardware appliqué à l'autonomie et au soin. Il s'inscrit dans une logique de plateforme collaborative, où les solutions sont partagées sous licences Creative Commons ou similaires, pour être reproduites librement.

Objectifs fonctionnels visés

- Permettre à tout utilisateur inscrit de publier un besoin d'aide technique ou une solution fabriquée.
- Mettre en relation les Makers volontaires avec les demandeurs.
- Proposer un catalogue de solutions consultables et téléchargeables (fichiers STL, PDF, images).
- Offrir un agenda des événements de la communauté (ateliers, webinaires, formations).
- Référencer les FabLabs de France via une carte interactive.
- Permettre la modération des contenus par un administrateur.
- Fournir un espace personnel à chaque utilisateur pour suivre ses projets.

Statut au moment de la transmission

Le site est en état pré-alpha. Cela signifie que les fondations architecturales sont posées et que la majorité des fonctionnalités décrites ci-dessus sont implémentées dans un premier état, mais qu'elles n'ont pas encore été validées en conditions réelles d'utilisation. Une bannière d'avertissement est d'ailleurs affichée en permanence aux visiteurs du site. Certaines tables SQL (signalements, evenements, etc.) sont gérées directement en SQL brut plutôt que via les entités Doctrine, ce qui représente une dette technique à traiter.

2. Architecture technique globale

Le projet suit une architecture MVC (Modèle - Vue - Contrôleur) standard, implémentée par le framework Symfony. Cette architecture sépare clairement les responsabilités : la logique métier est dans les contrôleurs, les entités représentent les données, et les templates Twig gèrent l'affichage.

Vue d'ensemble

Couche	Technologie	Rôle
Présentation	Twig + Bootstrap 5.3	Rendu HTML des pages
Interactivité front	Stimulus + Hotwire Turbo	Dynamisme sans SPA
Contrôleur	Symfony Controllers (PHP)	Traitement des requêtes HTTP
Logique métier	Services + Entités Doctrine	Règles métier et accès données
Persistance	Doctrine ORM + MySQL 8.0	Base de données relationnelle
Sécurité	Symfony Security Bundle	Auth, rôles, CSRF
Files d'attente	Symfony Messenger + Doctrine	Envoi d'emails asynchrones
Assets	AssetMapper (pas Webpack)	Gestion JS/CSS sans build

Choix architectural notable : AssetMapper

Le projet utilise le composant AssetMapper de Symfony plutôt que Webpack Encore. Ce choix signifie qu'il n'y a pas de pipeline de compilation JavaScript (pas de node_modules, pas de npm run build). Les dépendances JavaScript sont chargées directement via des URL CDN ou via le fichier importmap.php. C'est une approche plus légère, adaptée à un projet sans framework JavaScript complexe.

Flux d'une requête HTTP typique

1. L'utilisateur envoie une requête HTTP vers le serveur.
2. Le routeur Symfony (config/routes.yaml ou attribut #[Route]) identifie le contrôleur correspondant.
3. Le contrôleur accède aux données via Doctrine ORM (entités) ou via des requêtes SQL directes (Connection DBAL).
4. Le contrôleur prépare les variables et appelle le template Twig correspondant.
5. Twig génère le HTML final, renvoyé au navigateur.

3. Technologies utilisées et dépendances

Backend (PHP)

Package	Version	Usage
PHP	8.4 minimum	Langage de programmation principal
symfony/framework-bundle	8.0.*	Coeur du framework Symfony
doctrine/orm	^3.6	ORM pour la persistance des entités
doctrine/doctrine-bundle	^3.2	Intégration Doctrine/Symfony
doctrine/doctrine-migrations-bundle	^4.0	Gestion des migrations de schéma SQL
symfony/security-bundle	8.0.*	Authentification, autorisations, CSRF
symfony/form	8.0.*	Gestion et validation des formulaires
symfony/mailer	8.0.*	Envoi d'emails (config null en dev)
symfony/messenger	8.0.*	Traitement asynchrone des messages
symfony/twig-bundle	8.0.*	Moteur de templates
symfony/http-client	8.0.*	Appels HTTP externes (API FabLabs)
symfonycasts/verify-email-bundle	^1.18	Vérification d'email à l'inscription
symfony/stimulus-bundle	^2.32	Intégration Stimulus/Symfony
symfony/ux-turbo	^2.32	Hotwire Turbo pour les pages dynamiques
symfony/asset-mapper	8.0.*	Gestion des assets sans Webpack
symfony/monolog-bundle	^4.0	Logging de l'application

Frontend

- Bootstrap 5.3 : framework CSS chargé via CDN pour la mise en page et les composants.
- Bootstrap Icons 1.11 : librairie d'icônes SVG/police.
- Hotwire Turbo : navigation SPA-like sans rechargement complet de la page.
- Stimulus : framework JavaScript léger pour les contrôleurs front (CSRF notamment).
- Leaflet.js 1.9.4 : carte interactive pour l'affichage des FabLabs, chargé via CDN.
- OpenStreetMap : tuiles cartographiques utilisées par Leaflet.
- Google Fonts : polices Inter et Poppins chargées depuis les serveurs Google.

Infrastructure et outils de développement

- MySQL 8.0.32 : serveur de base de données (configuré dans .env).
- Symfony CLI : serveur de développement local recommandé.
- Composer 2 : gestionnaire de dépendances PHP.
- PHPUnit 12.5 : framework de tests unitaires (tests non encore écrits pour ce projet).
- Symfony Maker Bundle : génération de code (entités, contrôleurs, etc.) en développement.

4. Structure du projet et organisation des fichiers

Le projet respecte la structure standard d'une application Symfony. Voici les répertoires et fichiers les plus importants à connaître pour reprendre le travail.

Arborescence principale

Chemin	Description
src/Controller/	Contrôleurs PHP : logique de traitement des requêtes
src/Entity/	Entités Doctrine : mapping objet-relationnel
src/Repository/	Repositories Doctrine : requêtes personnalisées
src/Form/	Définition des formulaires Symfony
templates/	Templates Twig : HTML des pages
templates/base.html.twig	Layout principal (header, footer, nav)
templates/main/	Pages publiques du site
templates/security/	Pages connexion, inscription, espace
templates/admin/	Pages d'administration
config/packages/	Configuration des bundles Symfony
config/packages/security.yaml	Configuration de l'authentification
config/packages/doctrine.yaml	Configuration de la base de données
migrations/	Fichiers de migration SQL Doctrine
assets/	JavaScript et CSS sources
assets/app.js	Point d'entree JavaScript principal
public/	Racine web accessible (index.php, uploads)
public/uploads/solutions/	Fichiers uploadés par les utilisateurs
.env / .env.dev	Variables d'environnement (BDD, secret, etc.)
importmap.php	Déclaration des imports JavaScript (AssetMapper)
composer.json	Dépendances PHP et scripts Composer

Contrôleurs existants

- **MainController.php** : regroupe la quasi-totalité des routes publiques du site (accueil, solutions, besoins, agenda, fablabs, recherche, signalement, projets). C'est le fichier le plus volumineux, un refactoring en contrôleurs spécialisés est conseillé.
- **SecurityController.php** : gère l'inscription, la connexion, la déconnexion et la page espace personnel.
- **AdminController.php** : gère les routes d'administration (statistiques, signalements, suppression de contenus).

Templates Twig

- `base.html.twig` : le template parent. Il contient le header avec la barre de recherche, la navigation principale, les messages flash, et le footer. Toute modification globale de l'interface passe par ce fichier.
- `main/` : contient `index`, `solutions`, `solution_detail`, `besoins`, `besoin_form`, `solution_form`, `agenda`, `agenda_form`, `fablabs`, `recherche`, `membre`, `projets`, `signalement_form`, et les trois pages légales.
- `security/` : connexion, inscription, espace (tableau de bord utilisateur).
- `admin/` : statistiques et signalements.

5. Modèle de base de données

La base de données est MySQL 8.0. L'accès se fait principalement via Doctrine ORM pour les entités gérées, et directement via la couche DBAL (requêtes SQL brutes) pour les tables héritées de l'ancienne version du schéma.

Tables gérées par Doctrine (entités PHP)

Ces tables ont une entité PHP correspondante dans src/Entity/ et sont gérées proprement via l'ORM.

Table : utilisateurs

Cette table stocke tous les utilisateurs du site, quelle que soit leur rôle.

Colonne	Type	Description
UtilisateurID	INT AUTO_INCREMENT PK	Identifiant unique de l'utilisateur
Nom	VARCHAR(50)	Nom de famille
Prenom	VARCHAR(50)	Prénom
Email	VARCHAR(100) UNIQUE	Email, sert d'identifiant de connexion
Telephone	VARCHAR(255)	Numéro de téléphone (facultatif)
MotDePasse_Hash	VARCHAR(255)	Hash bcrypt du mot de passe
Role	ENUM(Membre, Maker, Soignant, Admin)	Rôle fonctionnel de l'utilisateur
Bio_Description	LONGTEXT	Description libre du profil
Competences_Techniques	LONGTEXT	Compétences techniques (Makers)
Region	VARCHAR(100)	Région de France
Consentement_Public	TINYINT(1)	1 si le profil est visible publiquement
Date_Inscription	DATETIME	Date d'inscription (auto-remplie)
pseudo	VARCHAR(255)	Pseudo affiché sur le site
accepte_contact_public	TINYINT(1)	Alias de Consentement_Public (doublon)

Table : projets

Représente les besoins exprimés par les membres. Le nom 'projets' est un héritage du schéma initial ; il correspond aux 'besoins' dans l'interface.

Colonne	Type	Description
ProjetID	INT AUTO_INCREMENT PK	Identifiant du besoin
Titre_Besoin	VARCHAR(100)	Titre court du besoin
Description_Detaillee	TEXT	Description complète
Statut	VARCHAR(50)	Ouvert / En cours / Terminé
Date_Creation	DATETIME	Date de publication du besoin
DemandeurID	INT FK -> utilisateurs	Auteur du besoin
maker_id	INT FK -> utilisateurs	Maker retenu pour ce besoin

Table : candidatures

Permet aux Makers de se porter volontaires sur un besoin. C'est la table de liaison entre projets et utilisateurs pour la mise en relation.

Colonne	Type	Description
id	INT AUTO_INCREMENT PK	Identifiant de la candidature
projet_id	INT FK -> projets	Besoin concerné
maker_id	INT FK -> utilisateurs	Maker candidat
date_candidature	DATETIME	Date de la candidature

Table : signalements

Stocke les signalements de contenus envoyés par les utilisateurs à la modération.

Colonne	Type	Description
SignalementID	INT AUTO_INCREMENT PK	Identifiant du signalement
UtilisateurID	INT FK -> utilisateurs	Auteur du signalement
TypeContenu	VARCHAR(50)	Type : Solution, Besoin, Commentaire, Evenement, Profil
ContenuID	INT	ID du contenu signalé
Raison	TEXT	Explication textuelle du signalement
Date_Signalement	DATETIME	Horodatage automatique

Tables gérées en SQL brut (sans entité Doctrine)

Ces tables existent dans la base de données et sont accessibles via la couche DBAL de Doctrine (requêtes SQL brutes via l'objet Connection). Elles n'ont pas d'entité PHP correspondante, ce qui signifie que Doctrine ne peut pas générer de migrations pour elles automatiquement. Une reprise propre du projet devrait créer les entités manquantes.

Table : solutions

Colonne	Type	Description
SolutionID	INT AUTO_INCREMENT PK	Identifiant de la solution
Titre_Solution	VARCHAR(100)	Titre de la solution
Description_Technique	TEXT	Description et instructions de fabrication
Licence	VARCHAR(50)	Licence de partage (Creative Commons, MIT...)
Materiel_Necessaire	TEXT	Liste du matériel requis
Difficulte_Fabrication	VARCHAR(50)	Très_Facile / Facile / Intermédiaire / Avancé
Date_Publication	DATETIME	Date de publication
Est_CoupDeCoeur	TINYINT(1)	1 si mise en avant par l'équipe
CreateurID	INT FK -> utilisateurs	Auteur principal
CoAuteurID	INT FK -> utilisateurs	Co-auteur (facultatif)

Table : fichiers

Fichiers attachés aux solutions (STL, PDF, images).

Colonne	Type	Description
FichierID	INT AUTO_INCREMENT PK	Identifiant du fichier
Nom_Fichier	VARCHAR(100)	Nom original du fichier
Chemin_URL	VARCHAR(255)	Chemin relatif depuis la racine web
Type	VARCHAR(20)	Extension en majuscules (STL, JPG, PDF)
Taille_Mo	DECIMAL(10,2)	Taille en mégaoctets
SolutionID	INT FK -> solutions	Solution à laquelle appartient le fichier
ProjetID	INT FK -> projets	Besoin auquel appartient le fichier (si applicable)

Table : commentaires

Commentaires déposés sur les solutions. La modération permet de masquer sans supprimer.

Colonne	Type	Description
CommentaireID	INT AUTO_INCREMENT PK	Identifiant du commentaire

Contenu_Texte	TEXT	Corps du commentaire
Date_Post	DATETIME	Date de publication
Est_Valide	TINYINT(1)	1 = visible, 0 = masqué par modération
AuteurID	INT FK -> utilisateurs	Auteur du commentaire
SolutionID	INT FK -> solutions	Solution commentée
ProjetID	INT FK -> projets	Besoin commenté (si applicable)

Table : evenements

Événements à venir (ateliers, webinaires, rencontres) proposés par la communauté.

Colonne	Type	Description
EvenementID	INT AUTO_INCREMENT PK	Identifiant de l'événement
Titre_Event	VARCHAR(100)	Intitulé de l'événement
Description	TEXT	Description détaillée
Date_Debut	DATETIME	Date et heure de début
Date_Fin	DATETIME	Date et heure de fin (facultatif)
Type	VARCHAR(50)	Webinaire / Atelier / Conférence / Rencontre / Formation
Lien_Externe_Organisateur	VARCHAR(255)	URL vers le site de l'organisateur

Table : messenger_messages

Table technique gérée automatiquement par Symfony Messenger pour le traitement asynchrone (envoi d'emails, etc.). Ne pas modifier manuellement.

6. Fonctionnalités implémentées

Catalogue de solutions

La page /solutions liste toutes les solutions publiées avec pagination (9 par page) et un filtre par niveau de difficulté. Chaque carte affiche le titre, la difficulté, la licence et l'auteur. Un clic mène vers la page de détail.

La page de détail /solutions/{id} affiche la description complète, le matériel nécessaire, les fichiers téléchargeables, et la section commentaires. Les commentaires peuvent être postés par tout utilisateur connecté. L'administrateur peut masquer ou supprimer les commentaires.

La création d'une solution (/solutions/nouvelle) propose un formulaire avec téléversement de fichiers. Les formats acceptés sont STL, JPG, JPEG, PNG et PDF, dans la limite de 10 Mo par fichier. Les fichiers sont stockés dans public/uploads/solutions/ avec un nom unique généré automatiquement.

Besoins communautaires

La page /besoins liste les besoins avec pagination et affiche le statut de chaque besoin (Ouvert, En cours, Terminé). Trois cas d'usage sont gérés dans l'interface :

- Le créateur du besoin voit la liste des candidats et peut en sélectionner un pour le valider.
- Un Maker connecté peut se porter volontaire sur le besoin d'un autre utilisateur.
- Si un Maker a déjà été retenu, une mention l'indique sur la carte.

Agenda

La page /agenda liste les événements à venir (filtrés par date $\geq$ aujourd'hui), triés du plus proche au plus lointain. Tout utilisateur connecté peut proposer un nouvel événement via /agenda/nouveau. Un lien vers le site de l'organisateur est obligatoire pour limiter les publications frauduleuses.

Carte des FabLabs

La page /fablabs consomme l'API publique de fablabs.io pour récupérer tous les laboratoires de fabrication référencés en France. Les données sont filtrées côté serveur (country_code FR et coordonnées GPS valides) puis passées au template. Côté navigateur, Leaflet.js affiche les marqueurs sur une carte OpenStreetMap. Un champ de recherche et un filtre par ville permettent de filtrer les résultats en temps réel sans rechargement de page.

Recherche globale

La page /recherche effectue une recherche textuelle simultanée dans les tables solutions (titre, description, matériel, nom/prénom auteur) et projets (titre, description, nom/prénom demandeur) via des requêtes SQL avec LIKE. Les résultats sont présentés en deux sections distinctes.

Espace personnel et tableau de bord

La page /espace (accessible uniquement après connexion) affiche les informations du compte connecté et des raccourcis rapides adaptés au rôle de l'utilisateur. La page /projets centralise : les besoins publiés par l'utilisateur avec le nombre de candidatures reçues, ses propres candidatures avec leur statut, et les solutions qu'il a publiées.

Profils publics

Chaque utilisateur ayant activé l'option Consentement Public dispose d'une page de profil publique accessible via /membre/{id}. Les informations affichées sont : pseudo, rôle, région, description, compétences. Pour les utilisateurs n'ayant pas consenti, seul le pseudo et le rôle sont visibles.

Signalement de contenus

Un bouton Signaler est disponible sur les solutions, les besoins, les événements, les commentaires et les profils. Le formulaire de signalement capture le type de contenu, l'identifiant du contenu et la raison. Les signalements sont listés dans l'interface d'administration avec des liens vers le contenu concerné.

Administration

L'interface d'administration est accessible via /admin/statistiques et /admin/signalements, exclusivement aux utilisateurs ayant le rôle Admin. Les fonctionnalités disponibles sont :

- Tableau de bord statistique : nombre total d'utilisateurs, de besoins publiés et de solutions partagées, liste des membres les plus actifs.
- Gestion des signalements : visualisation de tous les signalements avec aperçu du contenu visé, liens directs vers ce contenu, bouton de traitement (suppression du signalement).
- Suppression de solutions : supprime la solution, ses commentaires et ses fichiers associés.

- Suppression de besoins : supprime le besoin, ses candidatures, commentaires et fichiers.
- Suppression d'événements : suppression directe.
- Modération des commentaires : masquage/affichage et suppression définitive.

7. Système d'authentification et rôles

Authentification

L'authentification utilise le composant Security de Symfony. La configuration est dans `config/packages/security.yaml`. Le fournisseur d'utilisateurs (provider) est l'entité `App\Entity\Utilisateurs`, identifiée par son champ email. Le firewall principal utilise `form_login` avec les paramètres suivants :

- Page de connexion : `/connexion` (route `app_connexion`)
- Page de vérification du formulaire : `/connexion`
- Paramètre email du formulaire : `email`
- Paramètre mot de passe : `password`
- Redirection après connexion : `/espace` (route `app_espace`)
- Protection CSRF activée sur le formulaire de connexion

Hachage des mots de passe

L'algorithme `bcrypt` est utilisé via le `PasswordHasher` de Symfony. La configuration est dans `security.yaml` sous `password_hashers`. Lors de l'inscription, le mot de passe en clair (champ `plainPassword` non persisté) est haché via le service `UserPasswordHasherInterface` et stocké dans `MotDePasse_Hash`.

Rôles et contrôle d'accès

Symfony utilise une hiérarchie de rôles. L'entité `Utilisateurs` implémente `UserInterface` et retourne les rôles selon la valeur du champ `Role` :

Valeur en base	Rôles Symfony retournés	Description
Admin	<code>ROLE_ADMIN</code> , <code>ROLE_USER</code>	Accès complet dont l'interface d'administration
Maker	<code>ROLE_MAKER</code> , <code>ROLE_USER</code>	Peut proposer des solutions, se porter candidat
Soignant	<code>ROLE_SOIGNANT</code> , <code>ROLE_USER</code>	Accès standard avec mention soignant
Membre	<code>ROLE_MEMBRE</code> , <code>ROLE_USER</code>	Accès standard, peut exprimer des besoins
(autre ou null)	<code>ROLE_USER</code>	Accès de base après connexion

Les routes sont protégées via `access_control` dans `security.yaml` et via l'attribut `#[IsGranted]` sur les contrôleurs ou la méthode `denyAccessUnlessGranted()` en PHP. Les routes protégées actuellement configurées sont :

- /connexion et /inscription : PUBLIC_ACCESS (accessibles sans connexion)
- /projets et /espace : ROLE_USER
- /besoins/nouveau et /solutions/nouvelle : ROLE_USER
- /admin/* : ROLE_ADMIN (via attribut #[IsGranted] sur le contrôleur)

Protection CSRF

La protection CSRF est activée de deux manières : via Symfony Form (token automatique dans les formulaires gérés par le composant Form) et via des tokens manuels pour les formulaires HTML bruts (csrf_token() dans Twig, isCsrfTokenValid() dans les contrôleurs). Le composant Hotwire Turbo est configuré pour envoyer le token CSRF en header HTTP lors des soumissions de formulaires.

8. Interface utilisateur et design

Identité visuelle

Le site utilise un design system cohérent basé sur des variables CSS définies dans le template base.html.twig. Les couleurs principales sont :

Variable CSS	Valeur	Usage
--primary-color	#4F46E5	Couleur principale indigo, boutons, liens
--primary-dark	#4338CA	Variante sombre pour hovers
--primary-light	#818CF8	Variante claire
--secondary-color	#10B981	Vert emeraude, solutions, validations
--accent-color	#F59E0B	Jaune ambre, coups de coeur, mise en avant
--dark-color	#1F2937	Texte principal
--light-bg	#F9FADF	Fond de page

Polices

- Inter : police de corps de texte (poids 300, 400, 500, 600, 700, 800). Chargée depuis Google Fonts.
- Poppins : police de titres et sous-titres (poids 600, 700, 800). Chargée depuis Google Fonts.

Composants Bootstrap utilisés

- Système de grille (row/col) pour la mise en page responsive.
- Badges (badge) pour les statuts et les difficultés.
- Alertes (alert) pour les messages flash.
- Dropdowns pour le menu utilisateur connecté.
- Pagination pour les listes de solutions et de besoins.
- Cards pour les fiches solutions et besoins.
- Forms pour tous les formulaires.

Bannière pré-alpha

Une bannière jaune est affichée en permanence en haut de toutes les pages pour signaler que le site est en développement. Elle peut être fermée par l'utilisateur, et la fermeture est mémorisée en sessionStorage pour la durée de la session. Cette bannière est définie dans base.html.twig et doit être retirée lorsque le projet passera en beta ou en production.

Responsive

Le site est conçu pour fonctionner sur mobile, tablette et desktop grâce à Bootstrap. Des media queries spécifiques sont présentes dans app.css pour affiner le comportement sur les petits écrans. La taille minimale recommandée pour les éléments cliquables (44px) est mentionnée dans les commentaires CSS.

9. API externe et intégrations tierces

API FabLabs.io

La carte des FabLabs consomme l'endpoint public <https://www.fablabs.io/labs.json> qui retourne un tableau JSON de tous les FabLabs mondiaux enregistrés. L'appel est effectué côté serveur dans `MainController::fablabs()` via le composant `HttpClient` de Symfony (injection de `HttpClientInterface`).

Le filtrage sur `country_code = FR` et la vérification de la présence de coordonnées GPS sont faits en PHP. Les données filtrées sont ensuite encodées en JSON et injectées dans le template pour être consommées par le JavaScript de la carte Leaflet.

Points d'attention pour la reprise : l'API `fablabs.io` est une API publique non authentifiée, sans garantie de disponibilité. L'option `verify_peer` est désactivée dans le client HTTP, ce qui peut poser des problèmes de sécurité en production. Un mécanisme de mise en cache des résultats côté serveur devrait être ajouté pour éviter un appel HTTP à chaque chargement de page.

Envoi d'emails

Le composant Mailer de Symfony est configuré mais pointe vers un transport `null://null` en développement, ce qui signifie que les emails sont silencieusement ignorés. Le bundle `symfonycasts/verify-email-bundle` est installé pour une future fonctionnalité de vérification d'email à l'inscription, mais cette fonctionnalité n'est pas encore implémentée dans les contrôleurs.

Pour activer l'envoi d'emails en production, il faudra modifier la variable `MAILER_DSN` dans le fichier `.env.local` avec les paramètres d'un fournisseur SMTP (Mailtrap, Mailgun, SendGrid, serveur SMTP propre, etc.).

Messenger (traitement asynchrone)

Symfony Messenger est configuré avec un transport `doctrine://default` (stockage des messages dans la base de données MySQL). Le routing est configuré pour traiter de manière asynchrone les emails (`SendEmailMessage`) et les notifications (`ChatMessage`, `SmsMessage`). En développement, les messages sont traités de façon synchrone si le worker n'est pas lancé.

Pour traiter les messages en production, il faut lancer la commande : `php bin/console messenger:consume async --time-limit=3600`

10. Points d'attention et travail restant

Dettes techniques identifiées

- Entités manquantes : les tables solutions, fichiers, commentaires et evenements n'ont pas d'entité Doctrine correspondante. Toutes les opérations sur ces tables utilisent des requêtes SQL brutes via Connection DBAL, ce qui est moins maintenable et plus risqué (injections SQL si les paramètres ne sont pas correctement liés).
- MainController trop volumineux : ce contrôleur unique gère une vingtaine de routes différentes. Il devrait être découpé en contrôleurs spécialisés (SolutionController, BesoinController, AgendaController, FablabController).
- Migrations en doublon : les fichiers Version20260313101555.php et Version20260313102730.php exécutent exactement les mêmes opérations SQL. Cela crée une confusion dans l'historique des migrations.
- Colonne accepte_contact_public : l'entité Utilisateurs mappe à la fois Consentement_Public et accepte_contact_public sur la même colonne, avec des noms différents. Il y a une incohérence à clarifier.
- Pas de tests : le dossier tests/ ne contient que le bootstrap.php. Aucun test unitaire ni fonctionnel n'est écrit.
- Vérification d'email inactive : le bundle verify-email est installé mais non utilisé. Les utilisateurs s'inscrivent sans valider leur adresse email.
- Sécurité HTTP client : verify_peer = false dans l'appel à l'API FabLabs. En production, cette option doit être retirée ou remplacée par la configuration d'un certificat CA correct.

Fonctionnalités prévues mais non implémentées

- Vérification d'email à l'inscription (bundle installé, code manquant).
- Mise en cache de l'API FabLabs pour réduire les appels réseau.
- Système de notifications pour les demandeurs lorsqu'un Maker se porte candidat.
- Profil utilisateur modifiable (pas de formulaire d'édition du profil).
- Upload de photo de profil.
- Recherche avancée avec filtres combinés.
- Pagination de la recherche.
- Système de notation ou de validation des solutions.
- Contenu des pages légales (mentions légales, CGU, confidentialité) non finalisé.

Variables d'environnement à configurer

Pour tout nouvel environnement (développement, staging, production), les variables suivantes doivent être définies dans un fichier .env.local (non versionné) :

Variable	Description
APP_SECRET	Clé secrète de 32 caractères, utilisée pour les tokens CSRF et sessions

DATABASE_URL	DSN de connexion MySQL (host, port, nom BDD, utilisateur, mot de passe)
MAILER_DSN	Paramètres de l'envoi d'emails (smtp://, mailgun://, null://null pour dev)
APP_ENV	dev en développement, prod en production
DEFAULT_URI	URL de base du site pour la génération d'URL en CLI

11. Instructions de mise en service

Prérequis

- PHP 8.4 ou supérieur avec les extensions ctype et iconv.
- Composer 2.x.
- MySQL 8.0 ou supérieur.
- Serveur web (Apache, Nginx) ou Symfony CLI pour le développement local.

Installation en développement

6. Cloner le dépôt et se placer dans le répertoire du projet.
7. Créer le fichier .env.local avec les variables DATABASE_URL et APP_SECRET.
8. Installer les dépendances PHP : `composer install`
9. Créer la base de données : `php bin/console doctrine:database:create`
10. Exécuter les migrations : `php bin/console doctrine:migrations:migrate`
11. Lancer le serveur de développement : `symfony server:start` (ou `php -S localhost:8000 -t public/`)
12. Vider le cache si nécessaire : `php bin/console cache:clear`

Points de vigilance pour une mise en production

- Changer APP_ENV=prod dans .env.local ou les variables serveur.
- Compiler les assets : `php bin/console asset-map:compile`
- Supprimer les outils de développement (WebProfiler, DebugBundle) ou s'assurer qu'ils sont bien conditionnés à l'env dev via bundles.php.
- Configurer un vrai MAILER_DSN.
- Mettre en place HTTPS et retirer `verify_peer=false` dans HttpClient.
- Configurer le worker Messenger pour le traitement asynchrone des emails.
- Finir et faire valider les pages légales (mentions légales, CGU, politique de confidentialité) par un juriste.
- Retirer la bannière pré-alpha de base.html.twig.

Création du premier compte administrateur

Il n'existe pas de commande CLI pour créer un administrateur. La procédure actuelle est la suivante : s'inscrire normalement via /inscription, puis modifier manuellement le champ Role de l'utilisateur créé en base de données (`UPDATE utilisateurs SET Role = 'Admin' WHERE Email = 'votre@email.fr'`). Une future amélioration serait de créer une commande Symfony pour gérer cela proprement.

Nom de domaine et identité visuelle

Attention : le nom de domaine affiché dans le navigateur et les templates est 'Je Demain', mais le nom technique du projet dans les fichiers sources (namespace PHP, nom de base de données, répertoires) est toujours 'makegiver'. Une migration de nommage complète sera nécessaire si le nom 'Je Demain' est définitivement retenu. Cela concerne notamment : le namespace App (src/), le nom de la base de données dans DATABASE_URL, les commentaires et README du dépôt.

Rapport technique généré pour la reprise du projet Je Demain (MakeGiver)